

AI-Powered Student Workload Balancer — Full Project Blueprint

Course: UCS503P — Software Engineering **Team:** Khushi (1024030809), umang-gautam, vmahajan1, sgarg6 **Guide:** Dr. Nisha Thakur

1. Project Vision

A web app that generates and adaptively rebalances a student's study plan by combining:

A **deterministic scoring engine** (mastery + urgency → priority score per topic)

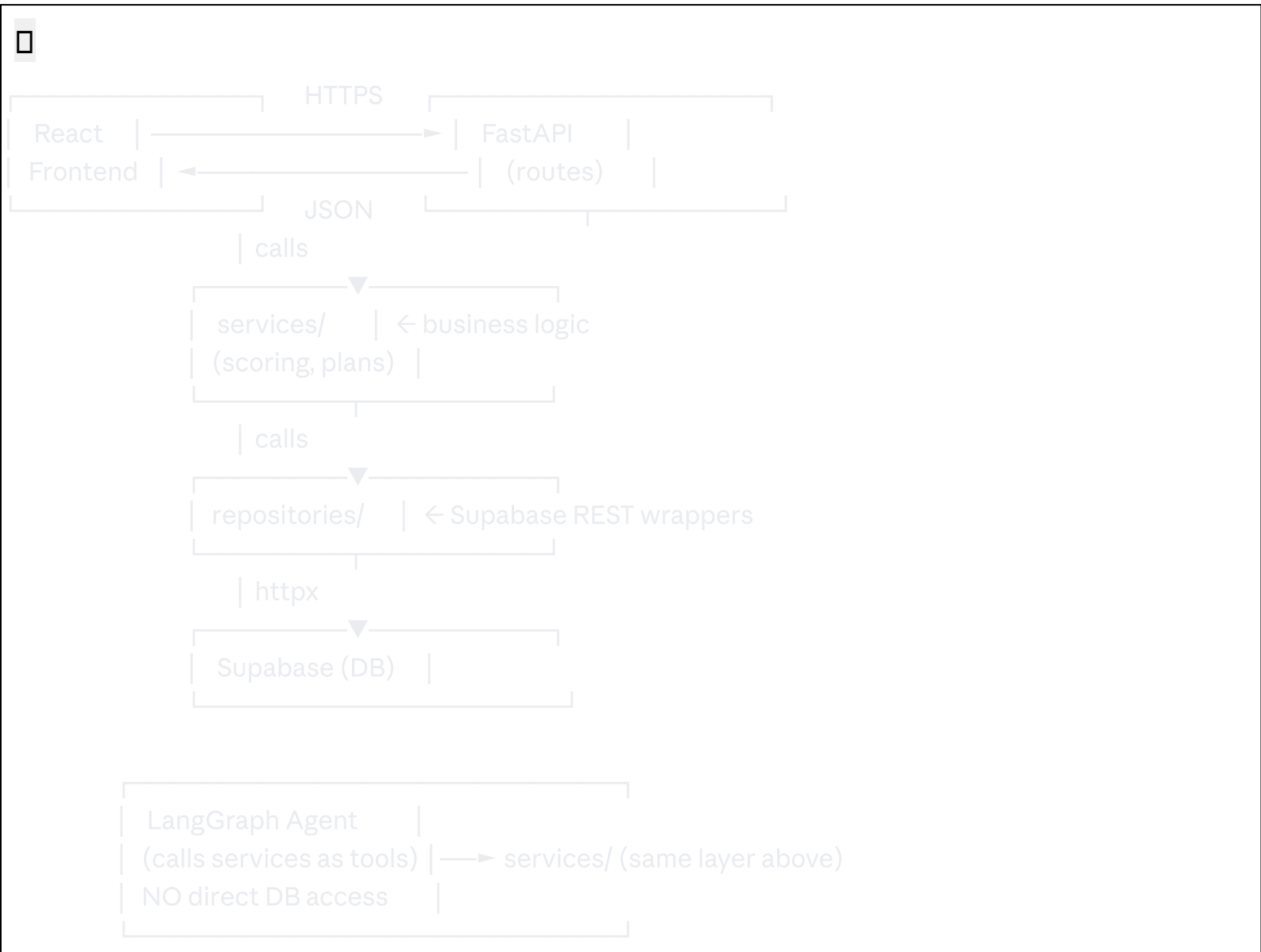
A **LangGraph reasoning agent** that adapts the plan over time (missed sessions, new low scores, new deadlines)

The agent **never touches the database directly** — it can only call service-layer tools. This is the core architectural safety guarantee and the strongest viva talking point.

2. Tech Stack

Layer	Choice	Why
Backend framework	FastAPI	async, auto docs, pydantic validation
Data access	Supabase REST API via <code>httpx</code>	no server/DB ops to manage, fast to iterate
Agent	LangGraph	explicit state machine for reasoning + tool calls
Frontend	React + Vite + Tailwind CSS	fast setup, utility-first styling
Containerization	Docker + docker-compose	reproducible dev/prod environment
CI	GitHub Actions	lint + test on every push
Deployment	Render/Railway (backend), Vercel (frontend)	free tiers, simple pipelines

3. System Architecture



Rule: routes → services → repositories → database. No layer skips ahead. Agent = tool-calls into services only.

4. Database Schema (Supabase, 8 tables)

Table	Purpose	Key columns
Student	User identity	id, name, email
Subject	Course catalog (shared, not per-student)	id, name, code
Enrollment	Student ↔ Subject link	student_id, subject_id
Topic	Sub-units of a Subject (shared, not per-Enrollment)	id, subject_id, name

Table	Purpose	Key columns
Assignment	Deadline/urgency tracker (NOT a mastery signal)	id, topic_id, due_date
PerformanceRecord	Multiple entries per topic over time	id, student_id, topic_id, score, recorded_at
StudyPlan	Umbrella container for a study period	id, student_id, start_date, end_date
StudySession	Individual scheduled study block	id, plan_id, topic_id, date, duration_minutes, status

Design decisions locked in:

Topic hangs off Subject, not Enrollment → no duplication across students

StudyPlan and StudySession are separate → plan is the umbrella, sessions are the grain, enabling adaptive rebalancing without recreating the whole plan

PerformanceRecord requires aggregation (avg first, recency-weighted later) since a topic has many records over time

5. Backend Folder Structure



```
backend/
├── app/
│   ├── main.py          # FastAPI app entrypoint
│   ├── core/
│   │   ├── config.py    # pydantic-settings, env vars
│   │   └── supabase_client.py # httpx client setup
│   ├── api/routes/
│   │   ├── students.py
│   │   ├── subjects.py
│   │   ├── topics.py
│   │   ├── assignments.py
│   │   ├── performance.py
│   │   ├── study_plans.py
│   │   └── agent.py      # endpoint(s) to trigger agent runs
│   ├── schemas/         # Pydantic request/response models
│   ├── services/
│   │   ├── scoring_service.py # mastery + urgency → priority score
│   │   ├── plan_service.py   # generates StudyPlan + StudySessions
│   │   └── agent_service.py  # LangGraph agent orchestration
│   ├── repositories/      # one file per table, httpx calls to Supabase REST
│   └── agent/
│       ├── graph.py       # LangGraph state machine definition
│       └── tools.py       # tool wrappers around services/
├── tests/
├── Dockerfile
├── requirements.txt
└── .env (never committed)
```

6. API Endpoints (v1)

Method	Path	Purpose
GET	<code>/health</code>	liveness check
CRUD	<code>/students</code> , <code>/subjects</code> , <code>/topics</code> , <code>/assignments</code>	basic resource management
POST	<code>/performance</code>	log a new performance record
GET	<code>/students/{id}/scores</code>	current priority scores per topic
POST	<code>/study-plans/generate</code>	build a new plan from current scores
GET	<code>/study-plans/{id}</code>	fetch plan + sessions

Method	Path	Purpose
PATCH	<code>/study-sessions/{id}</code>	mark session done/missed
POST	<code>/agent/rebalance</code>	trigger agent to adapt the plan

7. Scoring Engine Design

Mastery = average of a topic's `PerformanceRecord.score` values (simple average first; recency-weighted average as a future upgrade)

Urgency = derived from days remaining until the nearest `Assignment.due_date` for that topic (closer deadline → higher urgency)

Priority score = weighted combination of `(1 - mastery)` and urgency, normalized to **0-100**

Formula (starting point, tunable): `priority = 100 * (w1 * (1 - mastery_norm) + w2 * urgency_norm)` where `w1 + w2 = 1`

Lives entirely in `scoring_service.py`, pure functions, no I/O — easy to unit test

8. Study Plan Generation Logic

Input: scored topics (from `scoring_service`) + a time budget (e.g., hours/day, days until exam)

Output: `StudyPlan` row + ordered `StudySession` rows

Rule of thumb: sort topics by priority score descending, allocate session blocks until time budget exhausted, respect a minimum session length (e.g., 30 min)

9. LangGraph Agent Design

State: current plan, recent session outcomes, latest scores

Nodes: `detect_change` → `re-score` → `re-plan` → `explain_decision`

Tools available to agent: `get_scores()`, `get_plan()`, `update_plan()`, `log_session_outcome()` — all service-layer calls, never raw DB queries

Trigger conditions: missed session, new low performance record, new assignment added close to deadline

Test with mocked tool responses before wiring to real services

10. Frontend Structure (React + Vite + Tailwind)



```

frontend/
├── src/
│   ├── main.jsx
│   ├── App.jsx           # router setup
│   ├── api/
│   │   └── client.js      # axios instance, base URL, interceptors
│   ├── pages/
│   │   ├── Dashboard.jsx
│   │   ├── Subjects.jsx
│   │   ├── PerformanceEntry.jsx
│   │   └── StudyPlanView.jsx  # calendar/timeline (most complex UI piece)
│   ├── components/
│   │   ├── TopicCard.jsx
│   │   ├── SessionBlock.jsx
│   │   └── ScoreBadge.jsx
│   └── hooks/
│       └── useStudyPlan.js
├── tailwind.config.js
└── vite.config.js

```

Frontend build order:

Static shell + routing (no API calls yet)

Wire up CRUD pages against backend (Subjects, Performance Entry)

Dashboard showing scores ([ScoreBadge], [TopicCard])

Study Plan calendar/timeline view (hardest — do last)

Agent-triggered rebalance: refresh UI when plan updates

11. Integration Checklist

- ☐ CORS enabled on FastAPI for frontend origin
- ☐ Consistent error response shape (status + message) across all endpoints
- ☐ [.env.example] committed, real [.env] gitignored from commit #1
- ☐ Basic auth/session (even simple token) if multi-student use is in scope

12. Infra & Deployment

[Dockerfile] for backend, [docker-compose.yml] for local dev (backend + optionally a local Supabase emulation)

GitHub Actions: run [pytest] + lint on every push/PR

Deploy backend → Render or Railway (Docker image reused)

Deploy frontend → Vercel (auto-deploy from [main])

Environment variables (Supabase URL/key) set as secrets in both CI and deployment platform, never in code

13. Suggested Build Order (Phases)

Phase	Deliverable
0	Repo structure, <code>.gitignore</code> , REST/HTTP refresher if needed
1	FastAPI skeleton + <code>/health</code>
2	Supabase schema live, repositories layer (httpx CRUD)
3	Services + routes, full CRUD, pytest for services
4	Scoring engine (<code>scoring_service.py</code>) + unit tests
5	Study plan generation service + endpoint
6	LangGraph agent (tools, graph, mocked-tool tests)
7	React shell + routing + CRUD pages
8	Study plan calendar view + agent-triggered refresh
9	Docker + CI + deployment
10	Docs, architecture diagram, viva prep, journal entries

Parallelization tip: Phase 7 (basic React shell hitting CRUD endpoints) can start as soon as Phase 3 is done, in parallel with Phases 4–6, so frontend isn't a last-minute crunch.

14. Viva Talking Points (keep these sharp)

Agent can't bypass business logic — it has no DB access path, only service tools. This is a deliberate safety constraint, not an oversight.

Layered architecture — routes → services → repositories → DB, one direction only, easy to test each layer in isolation.

Why `Assignment` ≠ mastery signal — assignments are deadline/formality tracking; mastery comes purely from `PerformanceRecord` history.

Why `Topic` hangs off `Subject` — avoids duplicating shared curriculum content per student.

Why `StudyPlan`/`StudySession` are separate — enables rebalancing at the session grain without regenerating the whole plan.

15. Hygiene Rules (learned the hard way, don't repeat)

Never commit `.env` — check `.gitignore` before first commit, not after

Rotate credentials immediately if any secret leaks

Set `git config user.name`/`user.email` per-repo if your global identity doesn't match your GitHub account

Keep journal entries (`journals/<roll-no>-<name>/`) updated per phase, not in a batch at the end